

Towards Explainable Access Control

Citation for published version (APA):

Hasel Mehri, G., Morisset, C., & Zannone, N. (2025). Towards Explainable Access Control. In *SACMAT '25: Proceedings of the 30th ACM Symposium on Access Control Models and Technologies* (pp. 117-126). Association for Computing Machinery, Inc.. <https://doi.org/10.1145/3734436.3734439>

Document license:

CC BY

DOI:

[10.1145/3734436.3734439](https://doi.org/10.1145/3734436.3734439)

Document status and date:

Published: 07/07/2025

Document Version:

Publisher's PDF, also known as Version of Record (includes final page, issue and volume numbers)

Please check the document version of this publication:

- A submitted manuscript is the version of the article upon submission and before peer-review. There can be important differences between the submitted version and the official published version of record. People interested in the research are advised to contact the author for the final version of the publication, or visit the DOI to the publisher's website.
- The final author version and the galley proof are versions of the publication after peer review.
- The final published version features the final layout of the paper including the volume, issue and page numbers.

[Link to publication](#)

General rights

Copyright and moral rights for the publications made accessible in the public portal are retained by the authors and/or other copyright owners and it is a condition of accessing publications that users recognise and abide by the legal requirements associated with these rights.

- Users may download and print one copy of any publication from the public portal for the purpose of private study or research.
- You may not further distribute the material or use it for any profit-making activity or commercial gain
- You may freely distribute the URL identifying the publication in the public portal.

If the publication is distributed under the terms of Article 25fa of the Dutch Copyright Act, indicated by the "Taverne" license above, please follow below link for the End User Agreement:

www.tue.nl/taverne

Take down policy

If you believe that this document breaches copyright please contact us at:

openaccess@tue.nl

providing details and we will investigate your claim.



Towards Explainable Access Control

[BlueSky Paper]

Gelareh Hasel Mehri
Eindhoven University of Technology
Eindhoven, The Netherlands
g.hasel.mehri@tue.nl

Charles Morisset
Durham University
Durham, UK
charles.morisset@durham.ac.uk

Nicola Zannone
Eindhoven University of Technology
Eindhoven, The Netherlands
n.zannone@tue.nl

Abstract

Access control (AC) systems play an important role in ensuring security by regulating how resources are accessed, protecting sensitive information, and maintaining system integrity. Their complexity arises not only from diverse policies and mechanisms but also from the involvement of multiple stakeholders, including resource owners, administrators, and end-users. Taking inspiration from explainable AI and explainable security, we define the first model of access control explainability, as a quality measure of the explanation graph constructed around the decisions made within the AC system. We then explore the literature to identify how existing work can be integrated as explanatory processes. Finally, we leverage our framework to articulate three open research challenges: the collection and interpretation of AC decisions, the effective construction of AC explanation graphs, and the definition of meaningful and computationally efficient explanation quality metrics.

CCS Concepts

• Security and privacy → Access control.

Keywords

Explainable Access Control, Explainability Framework, Policy Comprehension

ACM Reference Format:

Gelareh Hasel Mehri, Charles Morisset, and Nicola Zannone. 2025. Towards Explainable Access Control: [BlueSky Paper]. In *Proceedings of the 30th ACM Symposium on Access Control Models and Technologies (SACMAT '25)*, July 8–10, 2025, Stony Brook, NY, USA. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3734436.3734439>

1 Introduction

Access control (AC) systems play a pivotal role in ensuring security by regulating how resources are accessed, protecting sensitive information, and maintaining system integrity. However, the complexity of AC systems arises not only from the diverse range of policies and mechanisms, but also from the involvement of multiple stakeholders, including resource owners, administrators, and end-users. Each decision made by an AC system can significantly impact these stakeholders. For instance, the choice of an AC model may restrict the types of policies a resource owner can express, a poor design

decision might leave critical resources unprotected, and denying access could negatively affect the usability of the system.

In this blue-sky paper, we draw inspiration from recent advances in *Explainable Artificial Intelligence (XAI)* [1, 5] and the emerging field of *Explainable Security* [68] to argue for the need to conceptualize *Explainable Access Control (XAC)*. Explainability in AC can enhance transparency, facilitate better policy management, and provide meaningful feedback to end-users, ultimately improving trust and security.

We introduce a high-level abstract framework, where explainability is formalized through the notion of an *explanation graph* (Section 3) and a structured approach to generating explanations for access control decisions (Section 4). We also show how existing techniques can be mapped onto this framework to construct effective explanations (Section 4).

Our objective is to lay the groundwork for deeper reflection on the explainability of AC systems. To this end, we use our framework to outline three open research challenges in Section 5: (i) the collection and interpretation of AC decisions, (ii) the effective construction of AC explanation graphs, and (iii) the definition of meaningful and computationally efficient explanation quality metrics. By addressing these challenges, we aim to advance the state of AC systems and foster the development of more transparent and accountable security mechanisms.

2 Background

This section provides background on access control and introduces the notion of explainability.

2.1 Access Control

Access control is a fundamental security mechanism that regulates who can access resources within a system and under what conditions. It ensures that only authorized users or entities can perform designated actions, thereby safeguarding sensitive resources and information. Access decisions are typically governed by predefined policies that consider factors such as user identity, role, environmental context, and specific attributes. Common access control models include Discretionary Access Control (DAC), where resource owners assign permissions; Mandatory Access Control (MAC), which enforces strict rules set by a central authority; Role-Based Access Control (RBAC), which grants access based on predefined roles; and Attribute-Based Access Control (ABAC), which evaluates multiple attributes to determine access permissions dynamically. As access control systems become increasingly complex—incorporating dynamic contexts, collaborative environments, and evolving security threats—the need for explainability grows. Ensuring that all involved entities understand access decisions enhances compliance,



This work is licensed under a Creative Commons Attribution International 4.0 License.

SACMAT '25, Stony Brook, NY, USA

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1503-7/2025/07

<https://doi.org/10.1145/3734436.3734439>

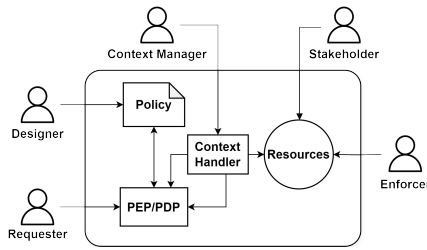


Figure 1: Access control system architecture and entities

Table 1: Access control system entities

Actor	Description
<i>Stakeholder</i>	(Co-)owns resources and monitors related access requests.
<i>Enforcer</i>	Ensures the system functions by applying access control policies.
<i>Designer</i>	Creates the policy enforced by the system.
<i>Context manager</i>	Manages the contextual information in the system.
<i>Requester</i>	Submits a request to access a resource and expects a decision.

facilitates auditing, and helps identify misconfigurations or policy inconsistencies. This section presents the main entities involved within an access control system and its development phases.

2.1.1 Entities. Several entities are involved within an access control system (Fig. 1). Table 1 summarizes these entities. The resource *stakeholder* (co-)owns the protected resources and defines the access requirements the system must enforce. The stakeholder also has visibility into access requests and the corresponding decisions made regarding their resources. The *enforcer* is responsible for processing access requests and making authorization decisions based on predefined policies. By correctly enforcing policies, the enforcer upholds system integrity and ensures compliance with stakeholder requirements. The *designer* plays a crucial role in defining and implementing the access control policies. This includes translating stakeholder requirements into technical configurations and ensuring that the system enforces them accurately and efficiently. To support dynamic decision-making, the *context manager* gathers and provides contextual information that influences access decisions. Factors such as user location, time of access, and device type help the system adapt to varying conditions and enforce policies accordingly. Finally, the *requester* is the entity that submits an access request for a specific resource, expecting an authorization decision based on the enforced policies and contextual conditions.

2.1.2 Development Phases. The Systems Development Life Cycle (SDLC) is a structured process that outlines key phases in system and product development to ensure efficient and cost-effective delivery [50]. For access control systems, it follows the general SDLC framework.

Specification: This phase is a critical step in access control SDLC, where stakeholders and policy enforcers collaborate to define the AC system's requirements. Stakeholders expect the system to align with their needs, including integration with existing infrastructure, cost-effectiveness, performance, delegation capabilities, privacy preservation, regulatory compliance, and service compatibility. The policy enforcer is responsible for gathering these requirements,

analyzing AC models, identifying limitations, and evaluating alternatives. The requirements must be formally documented and validated by stakeholders before progressing to the next phase.

Design: In this phase, the policy designer develops the overall structure of the AC model based on stakeholder requirements. The design outlines a detailed and specific policy, providing a holistic view of system implementation. The policy must satisfy key properties such as monotonicity, completeness, and attack resistance. Multiple designs may be proposed initially, with stakeholders reviewing and selecting the most suitable one for implementation.

Implementation: The implementation phase initiates the actual development of the system based on the designed policies. A well-structured design streamlines this process. The designer translates the detailed design into system components, defining interfaces and configuring mechanisms for authentication, access control, and logging. Security best practices, such as the principle of least privilege, are integrated to mitigate risks. Additionally, contextual factors are incorporated to refine access control policies.

Testing: After implementation, the testing phase ensures that the access control system and policies adhere to the specified requirements and function as intended. The system is evaluated to verify that access requests are correctly processed according to the defined policies, ensuring proper enforcement of authorization rules. The policy enforcer assesses whether the system grants or denies access appropriately, maintains security properties such as least privilege and separation of duties, and complies with regulatory and organizational requirements. If discrepancies or failures are identified, such as incorrect policy enforcement, unintended privilege escalation, or access violations, the system undergoes revision and refinement before deployment.

Deployment: During the deployment phase, the AC system is integrated into its operational environment. When a requester requests a specific resource, the system enforces the defined AC policies to determine whether access is granted. The system must be evaluated to ensure it meets access requirements and aligns with the initial specifications. In other words, it must demonstrate its ability to fulfill its intended purpose in its deployment environment.

Maintenance: During the maintenance phase, any issues that arise post-access, such as policy misconfigurations, access anomalies, or security breaches, are addressed. Stakeholders can review access logs to assess whether resource access aligns with the specified requirements and expectations. If discrepancies are identified, policy adjustments or system updates may be required to ensure compliance and security. Additionally, enhancements can be introduced to improve system performance or accommodate evolving access control needs, subject to stakeholder approval.

2.2 Explainability

Explainability refers to a system’s ability to help users understand its functioning [68]. Explanations offer insights into system operations and decision-making, enhancing user experience. Depending on the context and audience, they serve various purposes, such as increasing transparency and fostering trust. This section presents existing explainability frameworks relevant to our work.

Table 2: 6W’s framework for explainable security

Dimension	Description
<i>Who</i>	Actors participating in the system, both as providers and recipients of the explanations
<i>What</i>	Things that are explained
<i>Where</i>	Ways of making the explanations available
<i>When</i>	Stages in the system’s lifecycle to give the explanations
<i>Why</i>	Purposes for the explanations
<i>How</i>	Styles to communicate the explanations

2.2.1 6W’s Framework for Explainable Security. Inspired by advancements in explainable artificial intelligence (XAI) [57], Vigano and Magazzeni [68] explore the concept of *Explainable Security* (Xsec) as a fundamental approach to enhancing system understanding, trust, and management. Explainable security represents a complex, multi-faceted challenge involving multiple stakeholders, system models, diverse attack vectors, and corresponding defense mechanisms. The 6W’s framework of explainable security structures these challenges by addressing key questions across six dimensions: *Who*, *What*, *Where*, *When*, *Why*, and *How*. A summary of the 6W dimensions is presented in Table 2.

The first dimension, *Who*, identifies the key actors—or, as described, *dramatis personae*—involved in a system, both as providers and recipients of explanations. These include designers, users, and attackers, with a human or agent principal potentially assuming multiple roles. The *What* dimension focuses on the elements within the system that require explanation. The level of detail and purpose of these explanations vary by actor; for example, designers may require technical specifications, whereas users need justifications for decisions to build trust. The *Where* dimension determines where security explanations should be made accessible to relevant actors. These explanations can be embedded directly within the security mechanisms or provided separately, such as on a dedicated webpage. The *When* dimension specifies the stages of the system life cycle—design, implementation, or deployment—at which security explanations should be provided. The *Why* dimension addresses the purpose of explanations, such as establishing trust and ensuring verifiability, ultimately enhancing system security. Finally, the *How* dimension defines the methods through which explanations are delivered to their intended audience. These six fundamental questions collectively address the key aspects of providing effective security explanations. However, they are not entirely independent; rather, they are interconnected and influence one another [68].

2.2.2 Purposes for Explainability. The answer to the *Why* question in the 6W’s framework clarifies the benefits that explanations bring to a system. In [66], seven explanatory criteria are proposed to define such benefits by outlining the goals that explanations can achieve in recommender systems. Table 3 summarizes these goals.

The first goal is *transparency*, which provides a view of how the system operates and is closely linked to the established usability principle of “Visibility of System Status” [49]. Transparency enables the realization of other goals, such as *scrutability*. Scrutability allows users to correct the system’s reasoning and aligns with the usability principle of “User Control”. By providing explanations, users gain insight into the system’s decision-making process, enabling them to identify and rectify errors or misguided assumptions. *Trust* is also

Table 3: Explanation goals in recommender systems

Goal	Description
<i>Transparency</i>	Clarify how the system works
<i>Scrutability</i>	Allow users to tell the system it is wrong
<i>Trust</i>	Increase users’ confidence in the system
<i>Persuasiveness</i>	Convince users to try or buy
<i>Satisfaction</i>	Increase the ease of use or enjoyment
<i>Effectiveness</i>	Help users make good decisions
<i>Efficiency</i>	Help users make decisions faster

associated with transparency, as increased system visibility and user interaction enhance confidence in the system. The accuracy of the system also plays a crucial role in shaping user trust.

Explanations may also serve to persuade users to accept the system’s output, such as encouraging them to try or purchase a recommended product. This goal, referred to as *persuasiveness*, prioritizes benefits for the system rather than the user. Closely related to persuasiveness is *satisfaction*, which reflects the user’s acceptance of the overall system and has been found to correlate positively with longer explanations. If explanations assist users in making better decisions—rather than merely persuading them to accept the system’s recommendations—then *effectiveness* is being achieved. Explanations help users filter out irrelevant options and identify more useful ones. Similar to trust, effectiveness is highly influenced by the accuracy of the system’s algorithm. Additionally, explanations can enhance decision-making speed, contributing to *efficiency*. One way to improve efficiency is by helping users understand the relationships between competing options.

These goals may interact, with certain explanatory attributes supporting multiple objectives. However, some goals may appear related without a direct causal link; for example, research indicates that increased transparency does not necessarily enhance trust [15]. Moreover, trade-offs between goals may be necessary, as explanations cannot always optimize all objectives simultaneously, often requiring a balance between competing priorities [66]. In access control systems, some of these goals align with the objectives of security explanations, while others are less relevant (see Section 4).

2.2.3 Frameworks for Explanations. As Miller explained, “for millennia, philosophers have asked the questions about what constitutes an explanation, what is the function of explanations, and what are their structure”, and we refer to his seminal paper for a broad perspective [45]. Many established explanation models [3, 28, 71] follow communication principles [22] and can be expressed in conversational natural language or structured visual formats [45]. To reduce cognitive load [10, 13, 39], recent studies emphasize structured explanations over text-based descriptions, aligning with the idea of explanation graphs. Explanation graphs capture a model’s reasoning, enriching decisions with external or internal information. They help explain AI-based models, which are often opaque [18, 23, 60], or simply illustrate the reasoning behind a given decision [16, 73].

3 Explainable Access Control

In this section, we propose a general framework to characterize explainability in the context of access control. Our aim, in this blue-sky paper, is to propose a simple basis which can then be

Table 4: Example of decision set

ID	Decision
d_1	u_1 has accessed f_1
d_2	u_2 changed the access rights of f_1 to 777
d_3	The system is using the standard UNIX system.

extended, rather than providing a fully formed solution. We therefore introduce a set of definitions, illustrate them, and reflect on how they fit together. We choose to represent an explanation as a labeled directed graph, focusing on the idea that an explanation is a structured piece of knowledge that can be constructed and communicated, and take inspiration from that of knowledge graph [20]. We leave for future work the exploration of more specific formalisms, such as those mentioned in Section 2.2.3.

DEFINITION 1 (ACCESS CONTROL SYSTEM DECISION SET). *Given an access control system, we write $\mathcal{D}$ to denote the set of all possible decisions related to this access control system.*

Intuitively speaking, the access control decision set includes all decisions that can be collected across the different phases of the life cycle described in Section 2.1.2:

- **Specification decisions:** they include the choice of access control model (e.g., ABAC vs. RBAC), the selection of the access control relevant features (e.g., role, age, context), the choice of the implementation engine, etc. These decisions are likely to be kept as a textual knowledge base.
- **Design decisions:** they include the selection and update of the access control policy. Ideally, these decisions would be kept as a version control history on the structure of the policy, especially for centralized systems.
- **Implementation decisions:** they include interface technology choices (e.g., authentication, data storage) and, like specification decisions, are typically maintained as a textual knowledge base.
- **Testing decisions:** they include the coverage selected for testing (thus indicating possible gaps) and the decisions to change the system based on the outcome of the testing, which can be stored alongside design decisions.
- **Deployment decisions:** they include all runtime decisions made, including access decisions (e.g., permit/deny) and updates to the policy made (e.g., changing access rights). They would typically be collected from logs, such as access log or command history log.
- **Maintenance decisions:** they can include changes to the system following an audit or a complaint made by a stakeholder or user, and might be stored alongside design decisions.

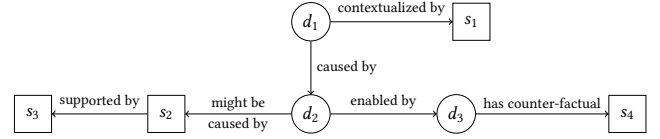
As an example, Table 4 lists some decisions in the context of a UNIX-based system. Clearly, these decisions can easily be collected: d_1 from an access log, d_2 from a bash history, and d_3 as an observation of the system.

DEFINITION 2 (EXPLANATION GRAPH). *Given an access control system, we write $\mathcal{G}$ for the set of explanation graphs, such that each graph is a pair $(\mathcal{S}, \mathcal{E})$, where $\mathcal{S} \supseteq \mathcal{D}$ is a set of access control statements containing at least the decisions, and $\mathcal{E} \subseteq \mathcal{S} \times \mathcal{R} \times \mathcal{S}$ is a set of directed labeled edges, where each edge is labeled with a relationship $r \in \mathcal{R}$.*

The set of statements includes at least the set of all possible decisions, but can also include other elements, such as:

Table 5: Example of explanatory statement

ID	Statement
s_1	In UNIX, a file with 777 means that any user can access it.
s_2	u_2 has been compromised.
s_3	f_1 is the only file on the system with 777 rights.
s_4	The system is not using SE Linux.

**Figure 2: Explanation graph for the decisions in Table 4 (represented by circles) and explanatory statements in Table 5 (represented by squares)**

- Description of a possible state of the system, such as the identification of an ongoing attack, which might indicate that some information is unavailable or corrupted.
- Counter-factual decisions, to explain which decisions have not been made, in contrast to those made.
- Output from a tool, such as a data table, a visualization image.
- Arguments to support a decision, such as a reference to source or a precedent.
- High-level definitions of security concepts.

DEFINITION 3 (EXPLANATORY PROCESS). *An explanatory process is a function $\tau : \mathcal{G} \rightarrow \mathcal{G}$, such that given an explanation graph $g = (\mathcal{S}, \mathcal{E})$, $\tau(g)$ is an explanation graph $(\mathcal{S}', \mathcal{E}')$ where $\mathcal{S} \subseteq \mathcal{S}'$ and $\mathcal{E} \subseteq \mathcal{E}'$.*

We treat an explanation graph as a knowledge base representing what is known about an access control system at a given time, with an explanatory process as a function that adds knowledge. We systematically explore explanatory processes from the literature in Section 4. Table 5 presents possible statements related to the decisions given in Table 4, as illustrated in Figure 2.

Analyzing the explanation graph can lead to new decisions and, consequently, a revised graph. For instance, in the previous example, the super-user might change the permissions on f_1 back to 700, thus preventing u_1 from accessing it, or start an investigation on u_2 , based on s_2 , possibly removing the user from the system. To prevent confusion, the graph should associate each decision and statement with a specific time frame.

DEFINITION 4 (EXPLANATION QUALITY METRICS). *Given an explanation graph $(\mathcal{S}, \mathcal{E})$, an explanation quality metric is a pair $(\mathcal{V}_\mu, \mu : \mathcal{G} \times \mathcal{S} \rightarrow \mathcal{V}_\mu)$ such that $\mathcal{V}_\mu$ is a value space and for any graph g and statement $s \in \mathcal{S}$, $\mu(g, s)$ represents the value associated with the statement s in the graph g .*

As mentioned in Section 2.2.3, there is a wide range of approaches to define and assess explanations. In the context of an explanation graph, some examples of metrics include:

- The inclusion of citations to trusted or whitelisted sources.
- The inclusion of well known security principles (e.g., least privilege or zero-trust).
- The assessment of the logical narrative construction.

- The analysis of the topology of the graph, for instance to show the absence of cycle (and thus avoiding circular explanations).
- User satisfaction or mental model understanding of the graph, as suggested in [68].

For instance, a topology metric on the example graph given above could be to ensure that each decision has a causal structure:

$$\mu_c(s, g) = \begin{cases} 1 & \text{if } g \vdash s \leadsto_{\text{caused by}} s' \text{ and root}(s'), \\ 0 & \text{otherwise} \end{cases}$$

where $g \vdash s \leadsto_{\text{caused by}} s'$ is true if s equals s' or there is a path in g from s to s' where each edge is labeled with *caused by*, and *root* is a function identifying root causes, i.e., statements that do not need any further justification. If we define *root* to be true for s_1 and s_3 , since they are high level descriptions and factual information from the system, respectively, it follows that for all decisions, μ_c would return 0. Indeed, even though we know that d_1 is caused by d_2 , we have no causal structure for d_2 , nor d_3 . This could be remediated by transforming the edge *might be caused by* from d_2 to s_2 and establishing s_2 as a root cause, for instance by formally establishing that s_2 is true. Similarly, d_3 could be linked to a formal justification for that choice.

DEFINITION 5 (EXPLAINABILITY). *We say that the explainability of an access control system is directly related to the aggregated quality of the explanations that can be derived for each of the decisions.*

4 Building Explanations for Access Control

As discussed in Section 3, statements in the explanation graph provide insight into access control decisions. To systematically structure the knowledge conveyed by these statements, we adopt a framework similar to the 6W's (see Section 2.2.1) and analyze explanations across six dimensions. While we align with the original framework for most dimensions, we modify the interpretation of *Where*, focusing on the source of the explanation rather than the medium through which it is delivered. This shift emphasizes the origin of the explanation and the necessary information for constructing it. Additionally, to determine *When* an explanation is required, we refer to the access control development life cycle (see Section 2.1.2). The deployment phase aligns with the access phase in an access control system, during which requests are evaluated, while the maintenance phase corresponds to post-access actions. Table 6 summarizes how existing frameworks and tools for explainable access control align with our proposed framework. Table 7 further elaborates on the *How* dimension and clarifies the objectives of prior work, illustrating how they operationalize explanatory processes, i.e., how explanations are generated, structured, and presented.

Specification: The requirements of an access control system are defined during the specification phase. The primary entities involved in this phase are the *stakeholder* and the *enforcer*, although the *designer* and *context manager* may also contribute in developing explanations (entities involved indirectly are shaded gray in Table 6). A key challenge in this phase is ensuring that stakeholders' access requirements are well-defined. In some cases, stakeholders may have incomplete or contradictory needs, requiring clarification through explanations. Another common issue arises when stakeholders are unaware of the inherent limitations of certain access control models due to their expressiveness or complexity. For example, Beimel et

al. [8] compare the Situation-Based Access Control (SitBAC) and Context models, examining how well users understand and define access policies in each. Their findings suggest that SitBAC's ontological structure enhances comprehension, making it more suitable for complex scenarios than the logic-based expressions used in the Context model. Similarly, Park et al. [51] analyze various access control models, comparing key characteristics such as decision parameters and control targets to highlight their strengths and weaknesses. These comparative approaches can build the statements in the explanation graph, improving transparency and helping stakeholders make informed decisions. Additionally, a stakeholder must trust that the proposed system aligns with their access requirements. Karimi et al. [37] address this by adopting the Resource-Event-Agent (REA) framework, a formal modeling approach that represents both business and access control models using common building blocks. By structuring both domains in a unified manner and clarifying how business rules translate into access control policies, this approach also establishes a relationship between explanation statements, effectively linking them within the explanation graph. Since the specification phase occurs before the system is built, explanations must rely on external sources rather than the system itself. These sources include access control model standards, security requirement documentation, and stakeholder input [70].

Design & Implementation Phases: After defining the requirements, the access control system is designed and implemented. The primary entities involved in this phase are the *enforcer*, *designer*, and *context manager*, with *stakeholders* contributing to clarifying requirements as needed. Access policies should be deployed according to the design and seamlessly integrated with the existing infrastructure. A key challenge during this phase is implementing complex policies. Some studies propose graphical techniques that can serve as explanation statements, supporting designers in policy development and increasing transparency in system operations. For instance, Nergaard et al. [48] propose a graphical policy editor for XACML, grouping compatible blocks and operators while providing visual feedback to ensure correct placement. Many studies leverage Unified Modeling Language (UML) to transition from informal requirements to formal representations. Class diagrams are widely used to depict access control components and their relationships [31, 34, 64], while object diagrams capture policy constraints such as separation of duty [29, 43, 54]. Other works focus on behavioral aspects, using use case and interaction diagrams to describe system states [38, 52, 69]. In some cases, UML-based representations facilitate verification and validation during testing. Beyond graphical tools, statements can be constructed through systematic approaches that guide designers in analyzing requirements and defining policies. Qingfeng [27] proposes heuristics to identify access control elements and specify rules, streamlining policy implementation. Some studies directly implement models using language-based approaches, requiring human input only for refinement [58], or apply machine learning techniques to support testing [35]. Explanations during the design and implementation phases can be system-generated—clarifying system structure—or provided through external guidelines on policy design and deployment.

Testing: Access control systems and policies are verified against specified requirements. As in earlier phases, the primary entities

Table 6: Framework for explainable access control

WHEN	WHO	WHAT	WHY	WHERE	HOW
Specification	Stakeholder Enforcer Designer Context manager	– Ambiguous, incomplete, or contradictory access requirements – Clarification of complex models [8, 51] – Advantages of a proposed model over alternatives [8, 51] – Compliance of a proposed model with access requirements [37]	Proposed model is expected to meet the access requirements. – <i>Transparency</i>: ensuring the requirements and AC models are well understood – <i>Trust</i>: increasing confidence that the system meets the requirements – <i>Effectiveness</i>: helping to adopt a proper system	Detached from AC system	Formal language [37] Others [8, 51]
Design & Implementation	Enforcer Designer Context manager Stakeholder	– Methodology for designing the policies from requirements – Methodology for implementing complex policies – Valid values for resources and requesters' attributes	Policies are expected to be deployed and integrated. – <i>Transparency</i>: clarifying how policies are designed and implemented – <i>Effectiveness</i>: helping designers make well-informed design and implementation decisions – <i>Efficiency</i>: supporting design and implementation	Within AC system Detached from AC system	Graphical language [29, 31, 34, 38, 43, 48, 52, 54, 64, 69] Others [27, 35, 58]
Testing	Enforcer Designer Context manager Stakeholder	– Verification of implementations against design and identifying violations – Verification of AC model against specified access requirements and identifying violations	The model is expected to respect the intended properties. – <i>Scrutability</i>: identifying conflicts in the system – <i>Trust</i>: ensuring the policies adhere to expected properties	Within AC system Detached from AC system	Graphical language [29, 43, 52, 54] Formal language [2, 6, 24, 31, 33, 34, 38, 42, 58, 69] Natural language [29, 31] Others [27, 35, 65]
Access	Requester Enforcer Designer Context manager	– Reasons for a denied request [21] – Adjustments to a denied request to obtain access [11, 21, 36] – Attributes of requester or requested resource relevant to a denied request	Requester expects access to be authorized. – <i>Transparency</i>: helping requester understand why their request was denied – <i>Trust</i>: increasing requester's confidence in decisions – <i>Effectiveness</i>: helping requester make informed access requests – <i>Scrutability</i>: helping requester inform if his attributes in context handler are incorrect	Within AC system Detached from AC system	Formal language [11, 21, 36]
Post Access	Stakeholder Enforcer Designer Context manager	– Access status of resources [4, 9, 14, 19, 26, 44, 46, 53, 55, 56, 63, 72] – Reasons access is granted or denied [12, 30] – Unauthorized access and incidents	Some resources are expected [not] to be accessed by some requesters. – <i>Transparency</i>: clarifying reasons for access decisions – <i>Effectiveness</i>: helping make proper decisions in case of incidents	Within AC system Detached from AC system	Graphical language [4, 9, 14, 19, 26, 44, 46, 53, 55, 56, 63, 72] Formal language [47] Others [12, 30]
	Enforcer Designer Context manager Stakeholder Requester	– Logging extent for exploratory analysis – Frequency of applying context updates – Misconfigurations in policies or system [59] – Policy remediation or migration alternatives	Policies and system are expected to be maintained and improved. – <i>Transparency</i>: helping designers understand the functionality of the system – <i>Scrutability</i>: helping designer reflect on the system	Within AC system Detached from AC system	Others [59]
	Stakeholder Enforcer Designer Context manager	– Reasons for policies not being enforced in collaborative authorization [17, 25, 32, 41]	Stakeholder expect their policy to be enforced. – <i>Transparency</i>: helping stakeholder understand the reasons for policy enforcement – <i>Trust</i>: ensuring stakeholder's confidence in policy enforcement – <i>Satisfaction</i>: increasing stakeholder's satisfaction with the overall AC system	Within AC system Detached from AC system	Natural language [17, 25, 32, 41]

involved in testing are the *enforcer*, *designer*, and *context manager*. Beyond supporting design and implementation, graphical UML techniques aid in constructing statements that facilitate policy validation. Policy properties such as separation of duty and role hierarchy align with UML constraint concepts and can be expressed using the Object Constraint Language (OCL). These constraints are visualized through UML diagrams, such as object diagrams, to identify potential violations [29, 43, 52, 54]. In some cases, when constraints are violated, the system provides feedback on the root causes [29, 31], which can be interpreted as adding edges to the explanation graph. Some studies integrate formal verification with UML-based approaches. For instance, works such as [34, 69] translate specifications into the B method—starting from an abstract model—to support formal verification and refinements. This method generates proof obligations to ensure model correctness and justify access control decisions [2]. Other approaches use automated reasoning tools for policy verification. Hu et al. [31] introduce an RBAC Authorization Environment (RAE) tool based on SAT solving to automatically verify policy properties. Similarly, Hughes et al. [33] translate XACML policies into Boolean satisfiability problems for SAT-based analysis. Additional verification techniques include model checking [24, 38], process algebra for detecting policy violations [6], and software verification and validation (V&V) tools for static and runtime

checks [58]. Answer Set Programming (ASP) has also been applied to verify compliance with security principles [42]. Recent research explores machine learning-based verification. In [35], policies generated from natural language requirements are reconstructed and validated using a multi-class classifier, identifying errors such as incorrect decisions or missing subjects/resources. Similarly, [65] employs decision trees to detect policy anomalies, including inconsistencies and redundancies. As in earlier phases, explanations during testing can be generated within the system—by analyzing and observing its properties—or externally through formal analysis to validate correctness and compliance.

Access: The system enforces access control policies to determine whether incoming requests are granted or denied. The primary entities involved in this phase are the *requester* and the *enforcer*, with the enforcer responsible for providing explanations. Explanations during the access phase primarily revolve around the requester's expectation that their access is authorized. Studies such as [21] offer explanation statements that enhance user experience and trust by explaining access failures through causal analysis. Additionally, abductive reasoning can suggest adjustments to help requesters obtain the desired access more effectively [11, 21, 36]. In [11], an advisor module is triggered when conditions for granting access are unmet, guiding

Table 7: Techniques to support explainability

Explainability Technique	Aim	Development Phase
Graphical Language		
UML Modeling	Visualizing the structure and behavior of AC models [29, 31, 34, 38, 43, 52, 54, 64, 69]	Design & Implementation
Visualization	Visualizing violations of constraints [29, 43, 52, 54]	Testing
	Supporting the authoring and editing of AC policies [48]	Design & Implementation
	Visualizing enforced AC policies, comparing them to specified policies, and making adjustments [4, 9, 14, 19, 26, 44, 46, 53, 55, 56, 63, 72]	Post Access
Formal Language		
Resource Event Agent	Comparing AC models with access requirements [37]	Specification
SAT Solver	Automated verification of constraints [31, 33]	Testing
SMT Solver	Verification of AC policies against security properties [67]	Testing
Process Algebra	Verification of AC policies against intended policy specification [6]	Testing
B Method	Verification of AC models against constraints such as redundancies and inconsistencies [2, 34]	Testing
	Verification of AC models against intended policy specification [34, 69]	Testing
Software V&V	Verification and validation of AC system implementation [58]	Testing
Answer Set Programming	Automated verification against intended policy specifications [42]	Testing
Model Checker	Automated verification of AC policy properties [24, 38]	Testing
Abductive Logic	Identifying reasons behind an access request being denied and suggesting alternative means for access [11, 21, 36]	Access
Causal Analysis	Identifying reasons behind an access request being denied [21]	Access
	Measuring contributions of attributes to access decisions [47]	Post Access
Binary Decision Diagram	Searching paths that satisfy an access request [36]	Access
	Exploring evaluation of all possible access requests [47]	Post Access
Natural Language		
	Reporting causes for anomalies, e.g., violated constraint [29, 31]	Testing
	Informing policy enforcement issues in collaborative authorization and potential adjustments [17, 25, 32, 41]	Post Access
Others		
Manual Analysis	Human-led review and comparison of AC models [8, 51]	Specification
Heuristics	Systematically designing AC policies [27]	Design & Implementation
	Systematically refining policy anomalies, e.g., redundancies and conflicts [27]	Testing
LLM	Providing explicit implementation of AC policies [35, 58]	Design & Implementation
Classification	Verifying AC policies against intended specifications to identify anomalies [35, 65]	Testing
XAI	Human-interpretable reasoning (by e.g., EBMs and LIME) for access decisions [12, 30]	Post Access
Log Audit	Tracing access events and investigating potential misuse [59]	Post Access

requesters on actions needed to obtain the required access. Similarly, Kapadia et al. [36] employ Binary Decision Diagrams (BDDs) to store access control rules and generate recommendations when access is denied. By analyzing alternative paths in the BDD that satisfy access rules, the system suggests modifications that may lead to successful authorization. Explanations may also incorporate contextual details to help identify inconsistencies in policy enforcement. Explanations can be provided to requesters either as part of the decision itself or upon request. Explanations justifying access decisions can be derived from the system’s internal reasoning. However, alternative suggestions for modifying the request can also be generated using external techniques independent of the system [21, 36].

Post Access: After policies are enforced, the system’s decisions and structure may be subject to review. The primary entities involved in explaining post-access phases depend on the explanation’s purpose. Some explanations clarify access decisions for *stakeholders*, *designers*, *context managers*, and *enforcers*, detailing how resources were accessed or restricted based on enforced policies. When explanations focus on system maintenance, limitations, and improvements, *stakeholders* are not directly involved. Conversely, in cases where policy enforcement across multiple stakeholders is explained, *designers* and *context managers* are indirectly engaged. These three explanation types in the post-access phase are distinguished in Table 6. Many works use graphical tools to visualize access decisions, building explanation statements that enable transparent comparison with intended policies. Some tools assist users who both own resources and configure policies. To avoid semantic errors caused by ambiguous privacy interfaces [19], some tools [4, 19] let users view their profiles from others’ perspectives to better understand

access implications in social networks. Other techniques go beyond policy enforcement visualization [44, 56, 72]. Mazzia et al. [44] enhance audience view techniques by representing users’ social networks, enabling them to adjust privacy settings for different groups. Wang et al. [72] introduce VeilMe, which visualizes personality traits from social media for privacy configuration. Pie-shaped visualization has been proposed to characterize users and media in a shared working space [56], assisting users in making informed sharing decisions. Further, some tools simplify policy configuration for non-experts, such as a layered graph-based tool [9] and the eye-metaphor user exposure tool [63]. Certain techniques assist designers in visualizing policy enforcement, particularly for handling complex or error-prone cases. For example, Cao et al. [14] introduce a matrix-based interface that simplifies the complexity of directly manipulating access control rules. Similarly, Reeder et al. [55] introduce an expandable grid to improve policy display and editing over list-based interfaces. Puchta et al. [53] evaluate visualization techniques for IAM systems and adopt a sunburst diagram to aid system structure analysis. To support security designers in managing XACML policies, Hasel Mehri et al. [26] adopt a node-link diagram with color-coding to illustrate policy evaluation results. Similarly, Morisset et al. [46] propose circle packing to visualize policies, allowing users to modify attributes and observe decisions. Beyond visualization, Morisset et al. [47] apply causal analysis to quantify attribute influence in ABAC decisions. Machine learning-based techniques can also utilize XAI methods to support decision-making in access control and clarify the reasons behind decisions. LIME [30] provides counterfactual explanations that resemble human reasoning. Explainable Boosting Machines

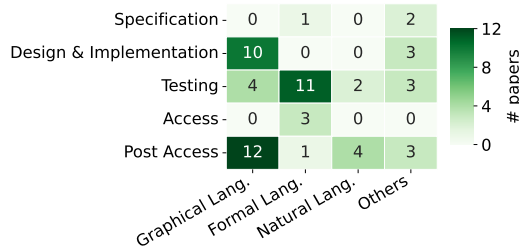


Figure 3: Distribution of explainability techniques across access control system development phases

(EBMs) [12] offer explanations by evaluating the contributions of individual features. These explanations can be either global, highlighting overall feature importance, or local, presenting specific factors that influence a particular decision. Explanation statements that focus on maintaining the system and facilitating potential improvements are provided to the designer and context manager. For example, Rostad et al. [59] demonstrate how examining access logs can help designers trace events, including bypassing normal RBAC through the use of exception mechanisms in their case. Audit trails should be accessible via a user-friendly interface for designers and contain enough detail to allow them to investigate undesired behavior. Finally, some statements aim to reassure stakeholders that the system enforces their intended policies—especially important in collaborative authorization and conflict resolution. Damen et al. [17] improve transparency by offering feedback when policies are not enforced. They explain when a policy is ignored due to an inapplicable rule or when the final decision differs from the stakeholder’s intent. Similarly, Mahmudlu et al. [41] detect mismatches between enforced decisions and stakeholder preferences, notifying affected stakeholders when mismatches arise. Hartog et al. [25] generate understandable justifications for overruled decisions, offering feedback tailored to the user’s role and the factors influencing the override. A large part of post-access explanation stems from describing the system’s decisions. A key source in this phase is log data, which records system activity and policy enforcement. Other external sources can include usage feedback and contextual information from monitoring systems.

5 Discussion & Conclusions

In this paper, we introduce access control explainability as a measure of quality on the explanation graph that can be built around the different decisions made within the access control system. We have shown that many existing techniques can be used to build that graph. In this section, we reflect on our approach by articulating open research problems that are related to the problem of explainability in access control.

On collecting AC decisions. In some systems, it might be hard or impossible to collect the relevant decisions. For instance, in decentralized systems, deployment decisions might not be automatically monitored. Design and implementation decisions might not have been kept. Furthermore, decisions might contain confidential or sensitive information, and storing or sharing them might therefore

require some additional data protection (e.g., anonymization or minimization). This raises the first open research problem: *Given an AC system, how to safely collect all decisions relevant to its explainability?*

On systematically building the graph. Assuming all decisions and statements can be collected, the next challenge is how to process and integrate all relevant explanatory statements in the graph. In principle, manually constructing such a graph for a specific decision is feasible—provided there is access to all relevant information and a clear understanding of the applicable explanatory processes. However, the availability and applicability of these processes vary across different phases of the system lifecycle. Fig. 3 illustrates this variation by classifying the explainability techniques covered in this paper. We observe that phases such as Design, Implementation, Testing, and Post Access offer a variety of techniques, whereas Specification and Access remain underdeveloped. For example, most existing techniques for the Specification phase rely on manual analysis and lack systematic tooling support. In addition, to ensure the credibility and robustness of explanation graphs, their construction should be grounded in established security principles. Notable examples include the principles proposed by Saltzer and Schroeder [61], Basin et al. [7], and the ASCAA framework [62]. While these principles provide a conceptual foundation, practical construction of an explanation graph introduces additional challenges. Real-world systems may involve overlapping or conflicting statements, and scalability becomes a concern as the graph grows in complexity. We therefore articulate the second open research problem as: *Given an AC system, how to systematically build a graph using the output of multiple explanatory processes?*

On defining meaningful and computational quality metrics. Assuming we can systematically build an explanation graph, the final challenge is to understand how explanations can be effectively communicated to the relevant stakeholders. As shown in Fig. 3, certain explainability techniques are more frequently proposed, while others remain underexplored. For example, techniques for the Access phase rely exclusively on formal languages. Although formal proofs may offer precision and structure, they are often insufficient for communicating explanations to diverse audiences without additional layers of interpretation or presentation [40]. Similarly, the graphical output of some explanatory processes may be technically accurate but require familiarity with the underlying tools or models to be useful. Ultimately, the effectiveness of explanations depends on their actionability for different stakeholders. It is essential to strike a balance between overly vague messages (e.g., a standard UNIX message like “Permission denied”) and oversharing explanations (e.g., “You need to log in as root with the password 123456 to access the file”). This leads us to our third open research problem: *Given an AC system, how to define explainability metrics that are adapted to the relevant stakeholders?*

To conclude this blue-sky paper, we have articulated for the first time the concept of explainability in access control. The graph-based approach in Section 3 is broad enough for extension yet specific enough to offer new insights. Similarly, the framework for explainability processes in Section 4 provides a comprehensive exploration of existing work within this perspective. Finally, we identified three open research challenges that lay the foundation for advancing explainable access control.

References

- [1] Abdelrahman Adadi and Mohammed Berrada. 2018. Peeking inside the black-box: A survey on explainable artificial intelligence (XAI). *IEEE Access* 6 (2018), 52138–52160.
- [2] Nasser Al-Hadhrani, Benjamin Aziz, Shantanu Sardesai, and Lotfi ben Othmane. 2015. Incremental Development of RBAC-Controlled E-Marking System Using the B Method. In *2015 10th International Conference on Availability, Reliability and Security*. 532–539.
- [3] C. Antaki and I. Leudar. 1992. Explaining in conversation: towards an argument model. *European Journal of Social Psychology* 22, 2 (1992), 181–194.
- [4] Mohd Anwar and Philip W. L. Fong. 2012. A Visualization Tool for Evaluating Access Control Policies in Facebook-Style Social Network Systems. In *Proceedings of the 27th Annual ACM Symposium on Applied Computing (SAC '12)*. ACM, 1443–1450.
- [5] Alejandro Barredo Arrieta, Natalia Díaz-Rodríguez, Javier Del Ser, Adrien Ben- netot, Siham Tabik, Alberto Barbado, Salvador Garcia, Sergio Gil-Lopez, Daniel Molina, Richard Benjamins, et al. 2020. Explainable artificial intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI. *Information Fusion* 58 (2020), 82–115.
- [6] Hamed Arshad, Ross Horne, Christian Johansen, Olaf Owe, and Tim A. C. Willemse. 2022. Process Algebra Can Save Lives: Static Analysis of XACML Access Control Policies Using mCRL2. In *Formal Techniques for Distributed Ob- jects, Components, and Systems*. Springer International Publishing, 11–30.
- [7] David Basin, Patrick Schaller, and Michael Schläpfer. 2011. *Applied information security: a hands-on approach*. Springer.
- [8] Dizza Beigel and Mor Peleg. 2010. The Context and the SitBAC Models for Privacy Preservation—An Experimental Comparison of Model Comprehension and Synthesis. *IEEE Transactions on Knowledge and Data Engineering* 22, 10 (2010), 1475–1488.
- [9] Anja Bertard and Jennifer-Kathrin Kopp. 2020. *Using Sugiyama-Styled Graphs to Directly Manipulate Role-Based Access Control Configurations*. 405–412.
- [10] Eliza Bobek and Barbara Tversky. 2016. Creating visual explanations improves learning. *Cognitive Research: Principles and Implications* 1 (12 2016). <https://doi.org/10.1186/s41235-016-0031-6>
- [11] P.A. Bonatti, E. Damiani, S. De Capitani di Vemercati, and P. Samarati. 2001. A component-Based architecture for secure data publication. In *Seventeenth Annual Computer Security Applications Conference*. 309–318.
- [12] Lucía Cabanillas Rodríguez, Juan Montes-Lopez, Diego López, and Pablo Serrano. 2025. DEBAC: Dynamic Explainable Behavior-Based Access Control. <https://www.sns-itrust6g.com/wp-content/uploads/2025/02/DEBAC-euCNC.pdf>
- [13] Huiying Cai and Xiaoqing Gu. 2019. Supporting collaborative learning using a diagram-based visible thinking tool based on cognitive load theory. *British Journal of Educational Technology* 50 (05 2019). <https://doi.org/10.1111/bjet.12818>
- [14] Xiang Cao and Lee Iverson. 2006. Intentional Access Management: Making Access Control Usable for End-Users. In *Proceedings of the Second Symposium on Usable Privacy and Security (SOUPS '06)*. ACM, 20–31.
- [15] Henriette Cramer, Vanessa Evers, Satyan Ramlal, Maarten Someren, Lloyd Rut- ledge, Natalia Stash, Lora Aroyo, and Bob Wielinga. 2008. The Effects of Trans- parency on Trust and Acceptance in Interaction with a Content-Based Art Rec- ommender. *User Model. User-Adapt. Interact.* 18 (2008), 455–496.
- [16] Han Cui, Shangzhan Li, Yu Zhang, and Qi Shi. 2023. Explanation Graph Gen- eration via Generative Pre-training over Synthetic Graphs. In *Findings of the Association for Computational Linguistics: ACL 2023*, Anna Rogers, Jordan Boyd- Graber, and Naoaki Okazaki (Eds.). Association for Computational Linguistics, Toronto, Canada, 9916–9934. <https://doi.org/10.18653/v1/2023.findings-acl.629>
- [17] Stan Damen, Jerry den Hartog, and Nicola Zannone. 2014. CollAC: Collaborative Access Control. In *2014 International Conference on Collaboration Technologies and Systems (CTS)*. 142–149.
- [18] Mauro Dragoni and Ivan Donadello. 2022. A knowledge-based strategy for XAI: The explanation graph. *Semantic Web Journal* (2022).
- [19] Serge Egelman, Andrew Oates, and Shriram Krishnamurthi. 2011. Oops, I did it again: mitigating repeated Access Control errors on facebook. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '11)*. ACM, 2295–2304.
- [20] Dieter Fensel, Umutcan Şimşek, Kevin Angele, Elwin Huaman, Elias Kärle, Olek- sandra Panasiuk, Ioan Toma, Jürgen Umbrich, Alexander Wahler, Dieter Fensel, et al. 2020. Introduction: what is a knowledge graph? *Knowledge graphs: Method- ology, tools and selected use cases* (2020), 1–10.
- [21] Sunil Kumar Ghai, Prateek Nigam, and Ponnurangam Kumaraguru. 2010. Cue: a framework for generating meaningful feedback in XACML. In *Proceedings of the 3rd ACM Workshop on Assurable and Usable Security Configuration (SafeConfig '10)*. ACM, 9–16.
- [22] H. P. Grice. 1975. Logic and conversation. In *Syntax and Semantics 3: Speech Arts*. Academic Press, New York, 41–58.
- [23] Jiuzhou Han, Wray Buntine, and Ehsan Shareghi. 2023. Reward Engineering for Generating Semi-structured Explanation. *arXiv preprint arXiv:2309.08347* (2023).
- [24] Frode Hansen and Vladimir Oleshchuk. 2005. Conformance Checking of RBAC Policy and its Implementation. In *Information Security Practice and Experience*, Robert H. Deng, Feng Bao, HweeHwa Pang, and Jianying Zhou (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 144–155.
- [25] Jerry Hartog and Nicola Zannone. 2016. Collaborative Access Decisions: Why Has My Decision Not Been Enforced? 109–130.
- [26] Gelareh Hasel Mehri, Tien Dung Le, Bram Cappers, Jerry Den Hartog, and Nicola Zannone. 2024. WiP: Enhancing the Comprehension of XACML Policies. In *Proceedings of the 29th ACM Symposium on Access Control Models and Technologies (SACMAT 2024)*. ACM, 41–46.
- [27] Qingfeng He and Annie Antón. 2009. Requirements-Based Access Control Anal- ysis and Policy Specification (ReCAPS). *Information and Software Technology* 51 (2009), 993–1009.
- [28] D. J. Hilton. 1990. Conversational processes and causal explanation. *Psychological Bulletin* 107, 1 (1990), 65–81.
- [29] Oliver Hoffrichter, Martin Gogolla, and Karsten Sohr. 2013. UML/OCL Based Design and Analysis of Role-Based Access Control Policies. In *Joint Proceedings of the First International Workshop On the Globalization of Modeling Languages (GEMOC 2013) and the First International Workshop: Towards the Model Driven Organization (AMINO 2013) Co-located with the 16th International Conference on Model Driven Engineering Languages and Systems (MODELS 2013)*, Miami, USA, September 29 - October 04, 2013 (CEUR Workshop Proceedings, Vol. 1102). CEUR-WS.org, 33–42.
- [30] R. Houtsma. 2022. An Explainable Machine Learning Approach to Risk-Adaptive Access Control. <http://essay.utwente.nl/89725/>
- [31] Hongxin Hu and Gail-Joon Ahn. 2010. Constructing authorization Systems using assurance management framework. *Trans. Sys. Man Cyber Part C* 40, 4 (2010), 396–405.
- [32] Hongxin Hu, Gail-Joon Ahn, and Jan Jorgensen. 2013. Multiparty Access Con- trol for Online Social Networks: Model and Mechanisms. *IEEE Transactions on Knowledge and Data Engineering* 25, 7 (2013), 1614–1627.
- [33] Graham Hughes and Tefvik Bultan. 2008. Automated Verification of Access Control Policies Using a SAT Solver. 10, 6 (2008), 503–520.
- [34] Faouzi Jaidi and Faten Labbene Ayachi. 2014. An Approach to Formally Validate and Verify the Compliance of Low Level Access Control Policies. In *2014 IEEE 17th International Conference on Computational Science and Engineering*. 1550–1557.
- [35] Sakuna Harinda Jayasundara, Nalin Asanka Gamagedara Arachchilage, and Gio- vanni Russello. 2024. RAGENT: Retrieval-Based Access Control Policy Generation.
- [36] Apu Kapadia, Geetanjali Sampemane, and Roy H. Campbell. 2004. KNOW Why Your Access Was Denied: Regulating Feedback for Usable Security. In *Proceedings of the 11th ACM Conference on Computer and Communications Security (CCS '04)*. ACM, 52–61.
- [37] Vahid Karimi, Paulo Alencar, and Donald Cowan. 2015. A Uniform Approach for Access Control and Business Models with Explicit Rule Realization. *International Journal of Information Security* 15 (2015).
- [38] Vahid R. Karimi, Paulo S. Alencar, and Donald D. Cowan. 2017. A Formal Modeling and Analysis Approach for Access Control Rules, Policies, and Their Combina- tions. *Int. J. Inf. Secur.* 16, 1 (2017), 43–74.
- [39] Jeff Klingner, Barbara Tversky, and Pat Hanrahan. 2011. Effects of visual and verbal presentation on cognitive load in vigilance, memory, and arithmetic tasks. *Psychophysiology* 48 (03 2011), 323–32. <https://doi.org/10.1111/j.1469-8986.2010.01069.x>
- [40] Leslie Lamport. 1994. How to write a long formula. *Formal aspects of computing* 6, 5 (1994), 580–584.
- [41] Rauf Mahmudlu, Jerry Hartog, and Nicola Zannone. 2016. Data Governance and Transparency for Collaborative Systems, Vol. 9766. 199–216.
- [42] Fabio Massacci and Nicola Zannone. 2008. A Model-Driven Approach for the Specification and Analysis of Access Control Policies. In *On the Move to Meaning- ful Internet Systems: OTM 2008*, Robert Meersman and Zahir Tari (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 1087–1103.
- [43] Tanjila Mawla and Maanank Gupta. 2024. Constraints Visualization and Specifica- tion for Activity-Centric Access Control. 371–380.
- [44] Alessandra Mazzia, Kristen LeFevre, and Eytan Adar. 2012. The PViz Compre- hension Tool for Social Network Privacy Settings. In *Proceedings of the Eighth Symposium on Usable Privacy and Security (SOUPS '12)*. ACM, Article 13, 12 pages.
- [45] Tim Miller. 2019. Explanation in artificial intelligence: Insights from the social sciences. *Artificial intelligence* 267 (2019), 1–38.
- [46] Charles Morisset and David Sanchez. 2018. On Building a Visualisation Tool for Access Control Policies. In *Information Systems Security and Privacy - 4th International Conference, ICISP 2018, Funchal, Madeira, Portugal, January 22- 24, 2018, Revised Selected Papers (Communications in Computer and Information Science, Vol. 977)*. Springer, 215–239.
- [47] Charles Morisset, Tim A. C. Willemse, and Nicola Zannone. 2019. A framework for the extended evaluation of ABAC Policies. *Cybersecur.* 2, 1 (2019), 6.
- [48] Henrik Nergaard, Nils Ulltveit-Moe, and Terje Gjøsøter. 2015. ViSPE: A Graphical Policy Editor for XACML. 107–121.
- [49] Jakob Nielsen and Rolf Molich. 1990. Heuristic evaluation of user interfaces. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Seattle, Washington, USA) (CHI '90). Association for Computing Machinery, New York, NY, USA, 249–256.

- [50] Oluwaseyi Olorunshola and Francisca Nonyelum Ogwueleka. 2021. Review of System Development Life Cycle (SDLC) Models for Effective Application Delivery. *Information and Communication Technology for Competitive Strategies (ICTCS 2020)* (2021).
- [51] Jaehong Park, Ravi Sandhu, Maanak Gupta, and Smriti Bhatt. 2021. Activity Control Design Principles: Next Generation Access Control for Smart and Collaborative Systems. *IEEE Access* 9 (2021), 151004–151022.
- [52] Aneta Poniszewska-Maranda. 2012. UML Representation of Extended Role-Based Access Control Model with the Use of Usage Control Concept. 131–145.
- [53] Alexander Puchta, Fabian Böhm, and Günther Pernul. 2019. *Contributing to Current Challenges in Identity and Access Management with Visual Analytics*. 221–239.
- [54] Indrakshi Ray, Na Li, Robert France, and Dae-Kyoo Kim. 2004. Using UML to Visualize role-Based Access Control Constraints. In *Proceedings of the Ninth ACM Symposium on Access Control Models and Technologies (SACMAT '04)*. ACM, 115–124.
- [55] Robert W. Reeder, Lujo Bauer, Lorrie F. Cranor, Michael K. Reiter, and Kami Vaniea. 2011. More than Skin deep: Measuring Effects of the Underlying Model on Access-Control System Usability. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '11)*. ACM, 2065–2074.
- [56] Jennifer Rode, Carolina Johansson, Paul DiGioia, Roberto Silva Filho, Kari Nies, David H. Nguyen, Jie Ren, Paul Dourish, and David Redmiles. 2006. Seeing Further: Extending Visualization as a Basis for Usable Security. In *Proceedings of the Second Symposium on Usable Privacy and Security (SOUPS '06)*. ACM, 145–155.
- [57] Avi Rosenfeld and Ariella Richardson. 2019. Explainability in human-agent systems. *Autonomous Agents and Multi-Agent Systems* 33, 6 (2019), 673–705.
- [58] Carlos E. Rubio-Medrano, Akash Kotak, Wenlu Wang, and Karsten Sohr. 2024. Pairing Human and Artificial Intelligence: Enforcing Access Control Policies with LLMs and Formal Specifications. In *Proceedings of the 29th ACM Symposium on Access Control Models and Technologies (SACMAT 2024)*. ACM, 105–116.
- [59] Lillian Røstad and Ole Edsberg. 2006. A Study of Access Control Requirements for Healthcare Systems Based on Audit Trails from Access Logs. 175–186.
- [60] Swarnadeep Saha, Prateek Yadav, and Mohit Bansal. 2022. Explanation graph generation via pre-trained language models: An empirical study with contrastive learning. *arXiv preprint arXiv:2204.04813* (2022).
- [61] J.H. Saltzer and M.D. Schroeder. 1975. The protection of information in computer systems. *Proc. IEEE* 63, 9 (1975), 1278–1308.
- [62] Ravi Sandhu and Venkata Bhamidipati. 2008. The ASCAA Principles for Next-Generation Role-Based Access Control. *Third International Conference on Availability, Reliability and Security*.
- [63] Roman Schlegel, Apu Kapadia, and Adam J. Lee. 2011. Eyeing Your Exposure: Quantifying and Controlling Information Sharing for Improved Privacy. In *Proceedings of the Seventh Symposium on Usable Privacy and Security (SOUPS '11)*. ACM, Article 14, 14 pages.
- [64] Andrey Sergeev and Raimundas Matulevicius. 2017. An Approach to Capture Role-Based Access Control Models from Spring Web Applications. In *2017 IEEE 21st International Enterprise Distributed Object Computing Conference (EDOC)*. 159–164.
- [65] Riaz Ahmed Shaikh, Kamel Adi, and Luigi Logrippo. 2017. A Data Classification Method for Inconsistency and Incompleteness Detection in Access Control Policy Sets. *Int. J. Inf. Secur.* 16, 1 (2017), 91–113.
- [66] Nava Tintarev and Judith Masthoff. 2011. *Designing and Evaluating Explanations for Recommender Systems*. Springer US, 479–510.
- [67] Fatih Turkmen, Jerry den Hartog, Silvio Ranise, and Nicola Zannone. 2017. Formal analysis of XACML policies using SMT. *Comput. Secur.* 66 (2017), 185–203.
- [68] Luca Vigano and Daniele Magazzeni. 2020. Explainable Security. In *2020 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*. IEEE, 293–300.
- [69] Inna Vistbakka and Elena Troubitsyna. 2018. Towards Integrated Modelling of Dynamic Access Control with UML and Event-B. *Electronic Proceedings in Theoretical Computer Science* 271 (2018), 105–116.
- [70] Suné von Solms and Annlizé Marnewick. 2019. Identifying Security Requirements Body of Knowledge for the Security Systems Engineer. In *Information Security Education. Education in Proactive Information Security*, Lynette Drevin and Marianthi Theodoridou (Eds.). Springer International Publishing, Cham, 59–71.
- [71] D. Walton. 2007. Dialogical models of explanation. In *Proceedings of the International Explanation-Aware Computing (ExaCt) Workshop*. 1–9.
- [72] Yang Wang, Liang Gou, Anbang Xu, Michelle X. Zhou, Huahai Yang, and Hernan Badenes. 2015. VeilMe: An Interactive Visualization Tool for Privacy Configuration of Using Personality Traits. In *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems (CHI '15)*. ACM, 817–826.
- [73] Wen Zhang, Bibek Paudel, Wei Zhang, Abraham Bernstein, and Huajun Chen. 2019. Interaction Embeddings for Prediction and Explanation in Knowledge Graphs. In *Proceedings of the Twelfth ACM International Conference on Web Search and Data Mining (Melbourne VIC, Australia) (WSDM '19)*. Association for Computing Machinery, New York, NY, USA, 96–104.